

GRAND CHALLENGE MONOGRAPH COLLECTION

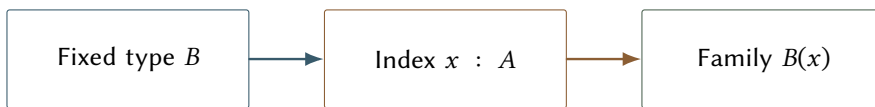
COMPREHENSION

Canonical Plate Folio

Volume II — Plates 1–42

Grand Challenge Labs

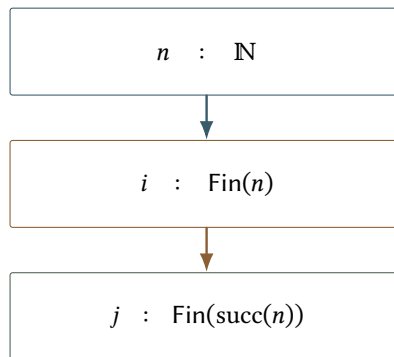
Fixed World / Dependent World



A family judgment permits type formation to depend on an earlier term.

Figure 1: **Plate 1. Fixed World / Dependent World.** A family judgment permits type formation to depend on an earlier term. The fiber language is geometric intuition; no set-theoretic model is assumed.

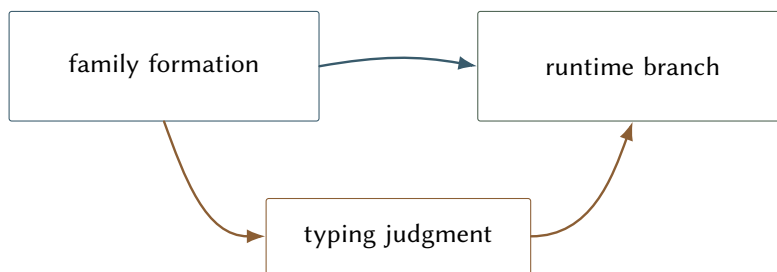
The Telescope



Context order carries dependency information.

Figure 2: **Plate 2. The Telescope.** Context order carries dependency information. Arrows represent free-variable dependency, not runtime dataflow.

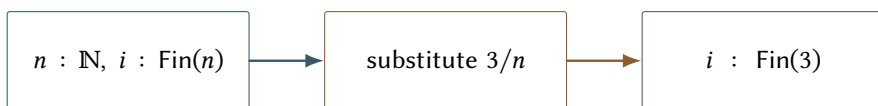
Dependency Is Static Structure



Type dependency is a formation discipline, not merely dynamic tagging.

Figure 3: **Plate 3. Dependency Is Static Structure.** Type dependency is a formation discipline, not merely dynamic tagging. Implementations may retain runtime evidence, but the judgments remain distinct.

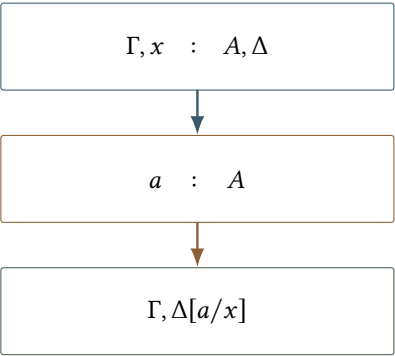
Substitution Rewrites the World



Substitution specializes later declaration types as well as terms.

Figure 4: **Plate 4. Substitution Rewrites the World.** Substitution specializes later declaration types as well as terms. This is structural substitution, not evaluation.

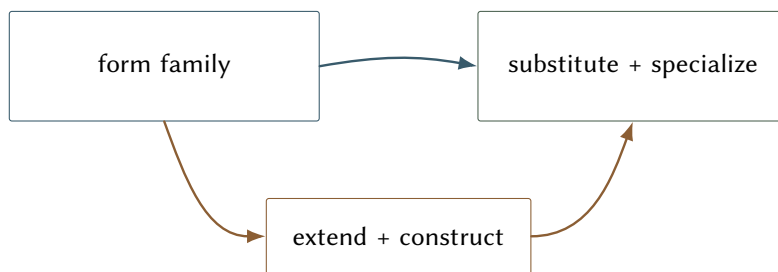
The Trailing Context



The generalized theorem must account for dependent declarations after the substituted variable.

Figure 5: **Plate 5. The Trailing Context.** The generalized theorem must account for dependent declarations after the substituted variable. Well-formedness side conditions remain explicit.

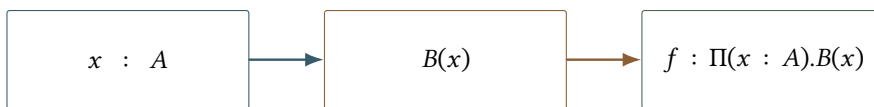
The Comprehension Loop



Family formation and substitution form a recurring compositional cycle.

Figure 6: **Plate 6. The Comprehension Loop.** Family formation and substitution form a recurring compositional cycle. Formal derivations are trees; the loop is pedagogical organization.

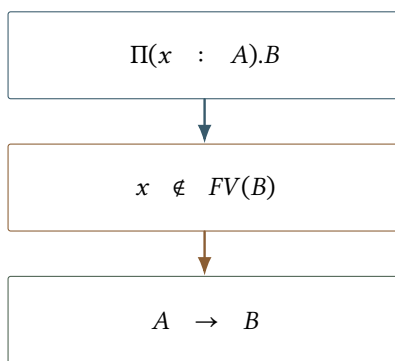
Pi as a Bundle of Codomains



A dependent function chooses an output in the codomain selected by its input.

Figure 7: **Plate 7. Pi as a Bundle of Codomains.** A dependent function chooses an output in the codomain selected by its input. The bundle picture is intuition, not a claim about one particular semantic model.

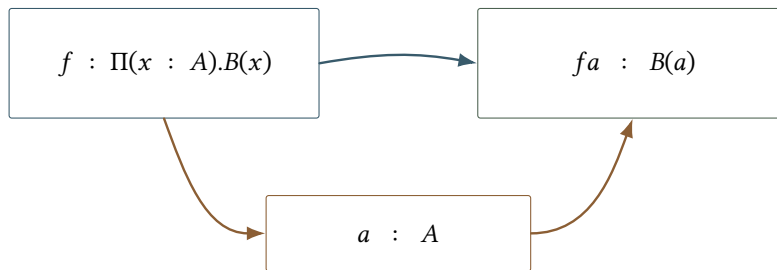
Constant Family: Pi Becomes Arrow



The ordinary arrow is the constant-family specialization of dependent product in this presentation.

Figure 8: **Plate 8. Constant Family: Pi Becomes Arrow.** The ordinary arrow is the constant-family specialization of dependent product in this presentation. Other formalisms may take arrow as primitive rather than abbreviation.

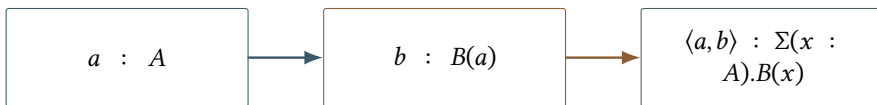
Application Specializes the Result



Dependent application substitutes the argument into the codomain.

Figure 9: **Plate 9. Application Specializes the Result.** Dependent application substitutes the argument into the codomain. Type substitution is not runtime mutation.

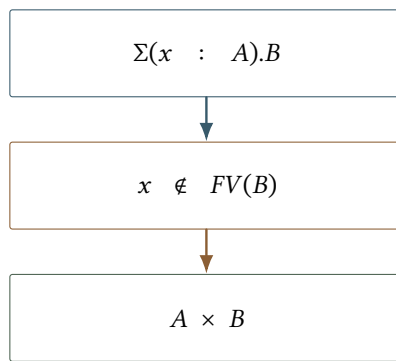
Sigma as a Fibered Pair



A dependent pair packages an index with
a payload in the selected family member.

Figure 10: **Plate 10. Sigma as a Fibered Pair.** A dependent pair packages an index with a payload in the selected family member. The fiber picture is semantic intuition only.

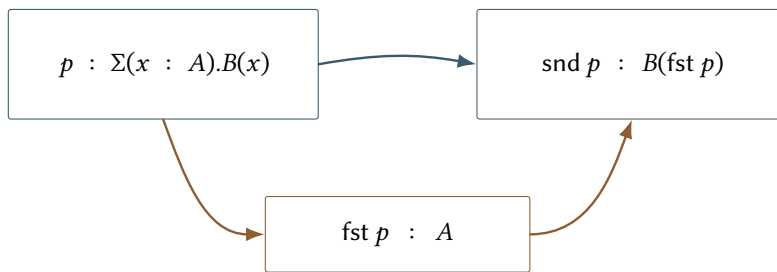
Constant Family: Sigma Becomes Product



An ordinary product is the constant-family
specialization of dependent sum here.

Figure 11: **Plate 11. Constant Family: Sigma Becomes Product.** An ordinary product is the constant-family specialization of dependent sum here. No cross-formalism claim of definitional identity is intended.

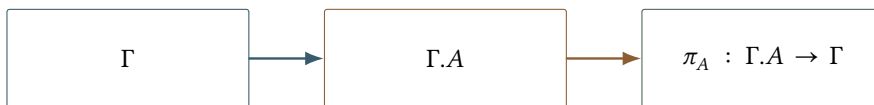
The Second Projection Remembers the First



The second projection has a type specialized by the first projection.

Figure 12: **Plate 12. The Second Projection Remembers the First.** The second projection has a type specialized by the first projection. Dependency is typed structure, not object-oriented field lookup.

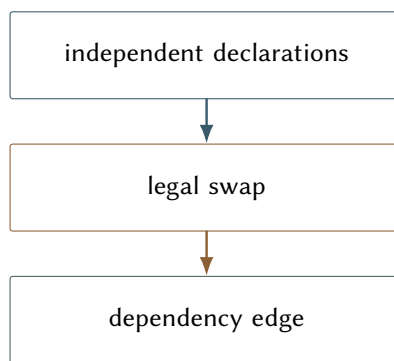
Context Extension Is Comprehension



A well-formed family extends a context
and induces a projection back to the base.

Figure 13: **Plate 13. Context Extension Is Comprehension.** A well-formed family extends a context and induces a projection back to the base. The arrow is a semantic reading of syntactic structure, not an extra runtime operation.

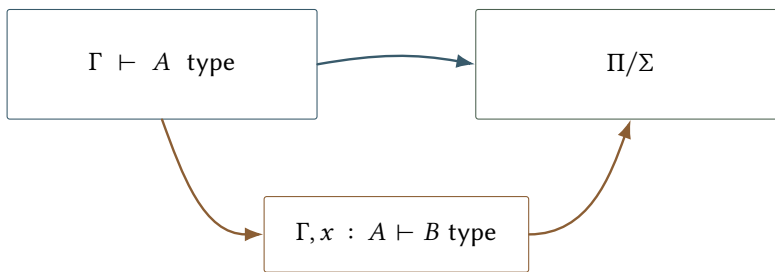
Exchange Is Conditional



Dependent contexts admit exchange only
when formation dependencies are preserved.

Figure 14: **Plate 14. Exchange Is Conditional.** Dependent contexts admit exchange only when formation dependencies are preserved. The diagram gives a sufficient condition, not every possible implementation optimization.

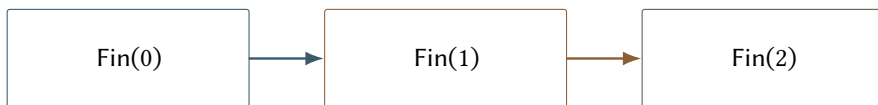
Pi and Sigma Consume the Same Family



Both major dependent formers are operations on the same family data.

Figure 15: **Plate 15. Pi and Sigma Consume the Same Family.** Both major dependent formers are operations on the same family data. Their introduction, elimination, and computation rules remain different.

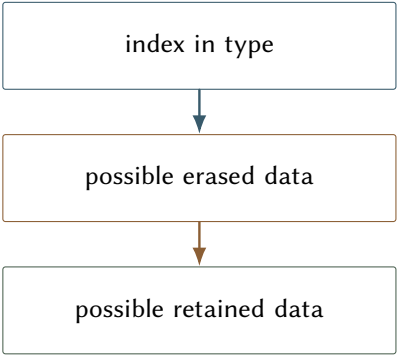
Fin as an Indexed Boundary



Constructor result indices make the finite bound part of admissible construction.

Figure 16: **Plate 16. Fin as an Indexed Boundary.** Constructor result indices make the finite bound part of admissible construction. The family encodes one bound invariant, not general program safety.

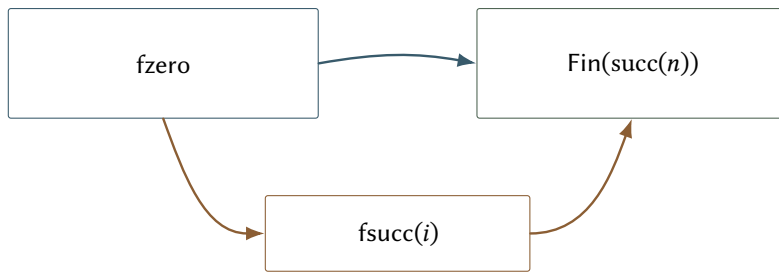
Static Index / Runtime Representation



Static role and runtime representation are distinct design questions.

Figure 17: **Plate 17. Static Index / Runtime Representation.** Static role and runtime representation are distinct design questions. The plate does not prescribe erasure or retention.

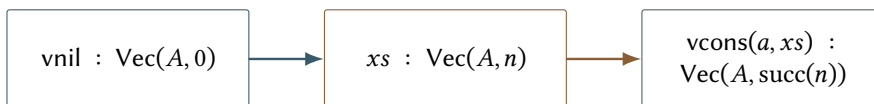
Canonical Fin Values



Finite-index values expose constructor shape constrained by the result index.

Figure 18: **Plate 18. Canonical Fin Values.** Finite-index values expose constructor shape constrained by the result index. Canonical forms classify values; they do not prove termination.

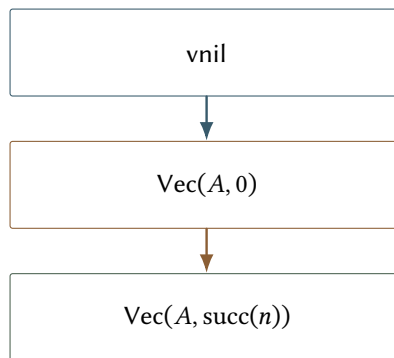
Vector Length in the Type



Vector constructors refine the length index by construction.

Figure 19: **Plate 19. Vector Length in the Type.** Vector constructors refine the length index by construction. Runtime layout may still coincide with ordinary lists.

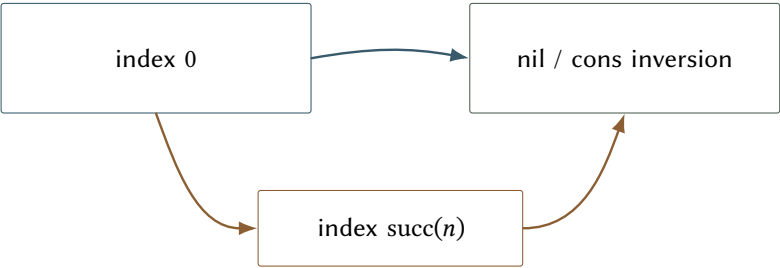
Impossible Constructor / Index Combinations



Result indices rule out constructor shapes inconsistent with the claimed length.

Figure 20: **Plate 20. Impossible Constructor / Index Combinations.** Result indices rule out constructor shapes inconsistent with the claimed length. The conclusion depends on the current definitional equality.

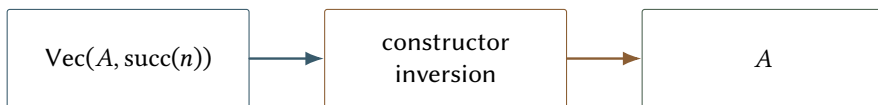
Shape from the Index



Canonical forms let an eliminator infer which constructor shapes remain possible.

Figure 21: **Plate 21. Shape from the Index.** Canonical forms let an eliminator infer which constructor shapes remain possible. This is inversion on values, not global evaluation behavior.

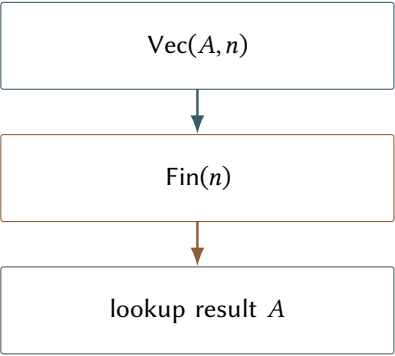
Vector Head Needs No Empty Branch



A nonempty vector type excludes the nil constructor before the head body is checked.

Figure 22: **Plate 22. Vector Head Needs No Empty Branch.** A nonempty vector type excludes the nil constructor before the head body is checked. The guarantee is local to the represented invariant.

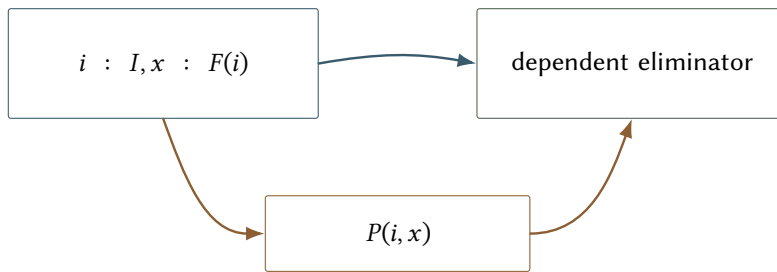
Safe Lookup Aligns Two Indices



Shared indices align vector shape with admissible positions.

Figure 23: **Plate 23. Safe Lookup Aligns Two Indices.** Shared indices align vector shape with admissible positions. The index does not certify external memory or foreign data.

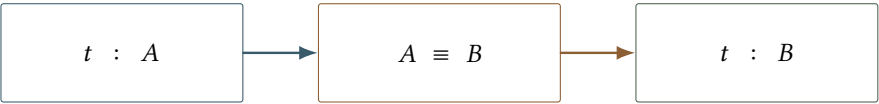
A Motive Preserves Dependency



A motive states the result family of dependent elimination.

Figure 24: **Plate 24. A Motive Preserves Dependency.** A motive states the result family of dependent elimination. The plate suppresses the full constructor-specific eliminator rules.

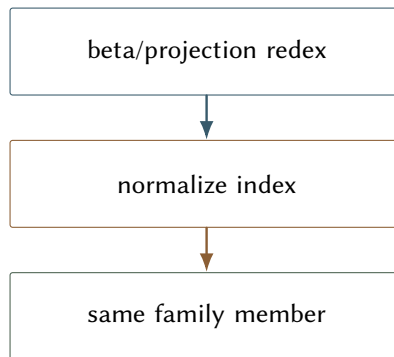
Conversion Through Definitional Equality



Conversion permits typing across equal types according to the kernel equality.

Figure 25: **Plate 25. Conversion Through Definitional Equality.** Conversion permits typing across equal types according to the kernel equality. Definitional equality is not arbitrary mathematical equality.

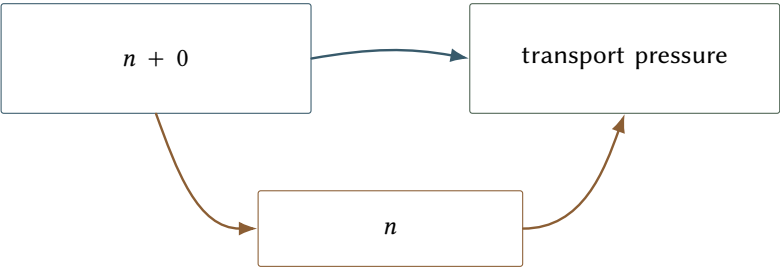
Computation Can Expose Equal Indices



Index computation can discharge type mismatches automatically.

Figure 26: **Plate 26. Computation Can Expose Equal Indices.** Index computation can discharge type mismatches automatically. Only the admitted computation rules contribute.

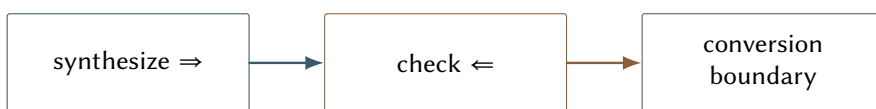
Where Definitional Equality Runs Out



Mathematical equality may exceed automatic definitional equality.

Figure 27: **Plate 27. Where Definitional Equality Runs Out.** Mathematical equality may exceed automatic definitional equality. Identity types and explicit transport are deferred to Volume VII.

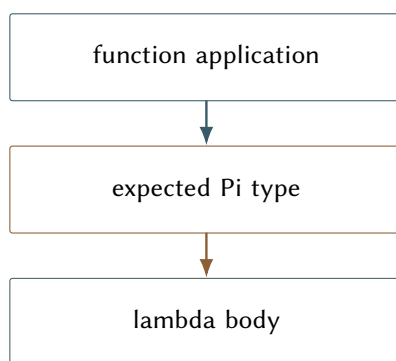
Bidirectional Information Flow



Bidirectional typing makes the direction of available information explicit.

Figure 28: **Plate 28. Bidirectional Information Flow.** Bidirectional typing makes the direction of available information explicit. It is an algorithmic presentation, not a new declarative truth relation.

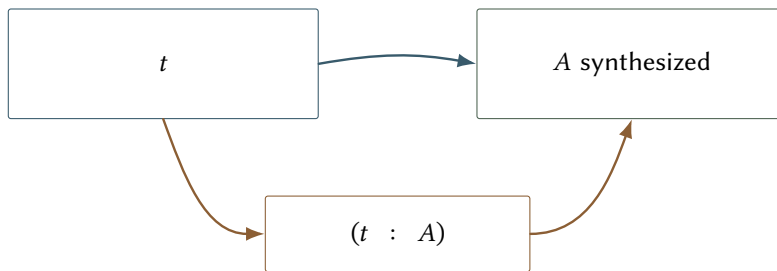
Application Synthesizes / Lambda Checks



Applications can synthesize specialized codomains while lambdas use expected domains.

Figure 29: **Plate 29. Application Synthesizes / Lambda Checks.** Applications can synthesize specialized codomains while lambdas use expected domains. More complete inference procedures are possible but not assumed.

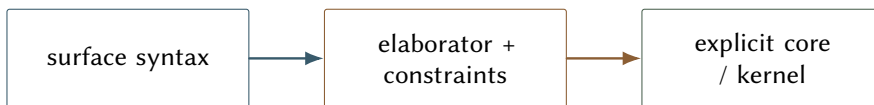
Annotations Move Information



An explicit annotation converts a checking obligation into a locally synthesizable type.

Figure 30: **Plate 30. Annotations Move Information.** An explicit annotation converts a checking obligation into a locally synthesizable type. Annotations are evidence for the checker, not proof of correctness by themselves.

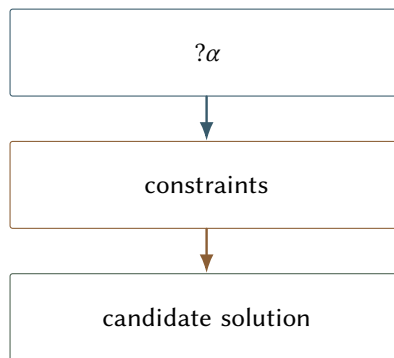
Elaborator and Kernel



Elaboration may be sophisticated while validity remains anchored in a smaller checker.

Figure 31: **Plate 31. Elaborator and Kernel.** Elaboration may be sophisticated while validity remains anchored in a smaller checker. Kernel acceptance does not automatically prove faithful surface intent.

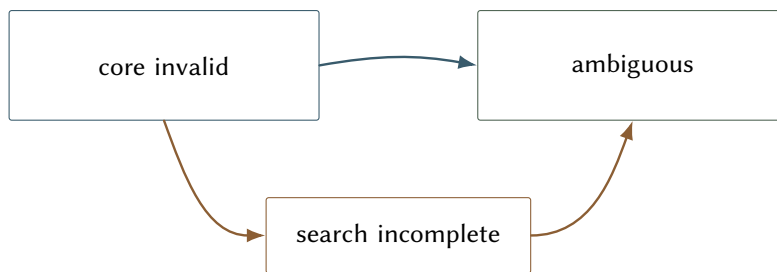
Metavariables Carry Constraints



A metavariable is a question constrained by uses, not an accepted proof term.

Figure 32: **Plate 32. Metavariables Carry Constraints.** A metavariable is a question constrained by uses, not an accepted proof term. The teaching solver handles only first-order syntactic constraints.

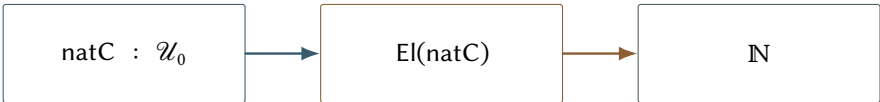
Three Elaboration Failures



Elaboration failure has distinct causes that diagnostics should not collapse.

Figure 33: **Plate 33. Three Elaboration Failures.** Elaboration failure has distinct causes that diagnostics should not collapse. A production elaborator may expose additional failure classes.

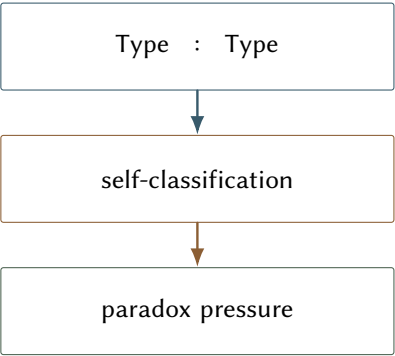
Codes and Decoding



Universe codes make descriptions of types
manipulable under controlled decoding.

Figure 34: **Plate 34. Codes and Decoding.** Universe codes make descriptions of types manipulable under controlled decoding. The code language is not the set of all types.

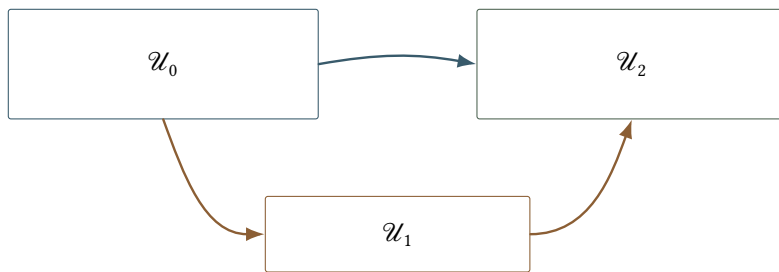
Why Type-in-Type Is Not Free



Unrestricted self-classification requires justification and is not silently admitted.

Figure 35: **Plate 35. Why Type-in-Type Is Not Free.** Unrestricted self-classification requires justification and is not silently admitted. The plate is a design warning, not a full paradox proof.

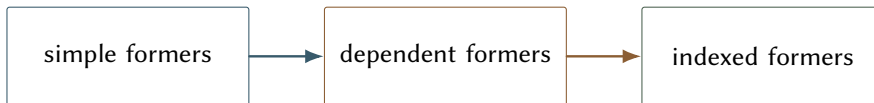
Stratification Pressure



Universe levels control how descriptions of types may themselves be classified.

Figure 36: **Plate 36. Stratification Pressure.** Universe levels control how descriptions of types may themselves be classified. Different type theories use different stratification policies.

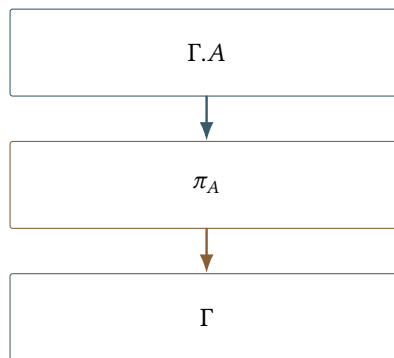
Formation Rules Build Worlds



Changing formation rules changes which computational specifications can be stated directly.

Figure 37: **Plate 37. Formation Rules Build Worlds.** Changing formation rules changes which computational specifications can be stated directly. Expressiveness does not automatically imply better engineering.

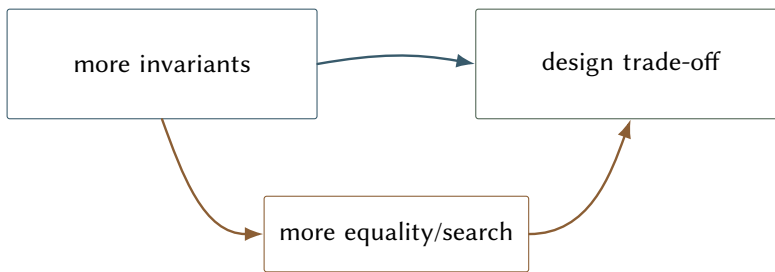
Comprehension as a Display Map



Categorical comprehension organizes context extension and reindexing.

Figure 38: **Plate 38. Comprehension as a Display Map.** Categorical comprehension organizes context extension and reindexing. A model validates syntax; syntax is not literally identical to one model.

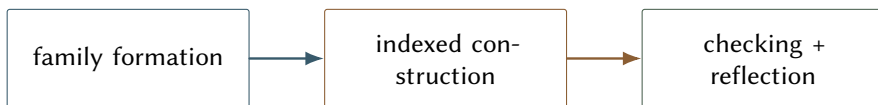
Expressiveness Has a Cost



Richer dependency trades specification power
against checking and elaboration burden.

Figure 39: **Plate 39. Expressiveness Has a Cost.** Richer dependency trades specification power against checking and elaboration burden. The balance is architecture-dependent, not a universal numeric law.

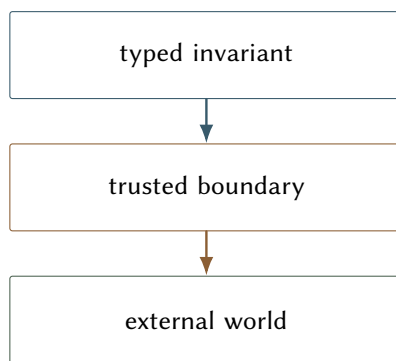
Volume II Synthesis



Dependency propagates from formation through construction, elimination, equality, and checking.

Figure 40: **Plate 40. Volume II Synthesis.** Dependency propagates from formation through construction, elimination, equality, and checking. The map is a conceptual dependency graph, not a proof implication graph.

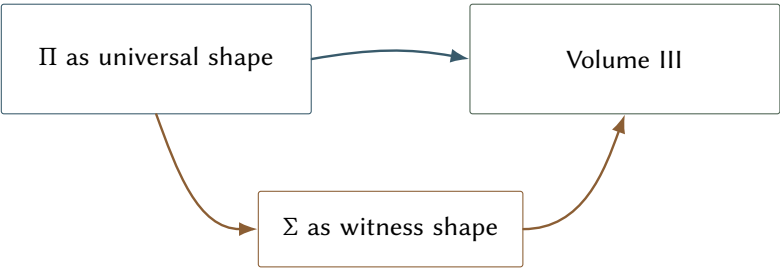
What Indices Cannot Guarantee



Typed indices guarantee only what is encoded and preserved inside their trust assumptions.

Figure 41: **Plate 41. What Indices Cannot Guarantee.** Typed indices guarantee only what is encoded and preserved inside their trust assumptions. Environmental and semantic correctness can require additional evidence.

Threshold to Proof / Program



Dependent construction creates the structural pressure for propositions-as-types.

Figure 42: **Plate 42. Threshold to Proof / Program.** Dependent construction creates the structural pressure for propositions-as-types. The logical correspondence is previewed here, not yet established as a theorem.